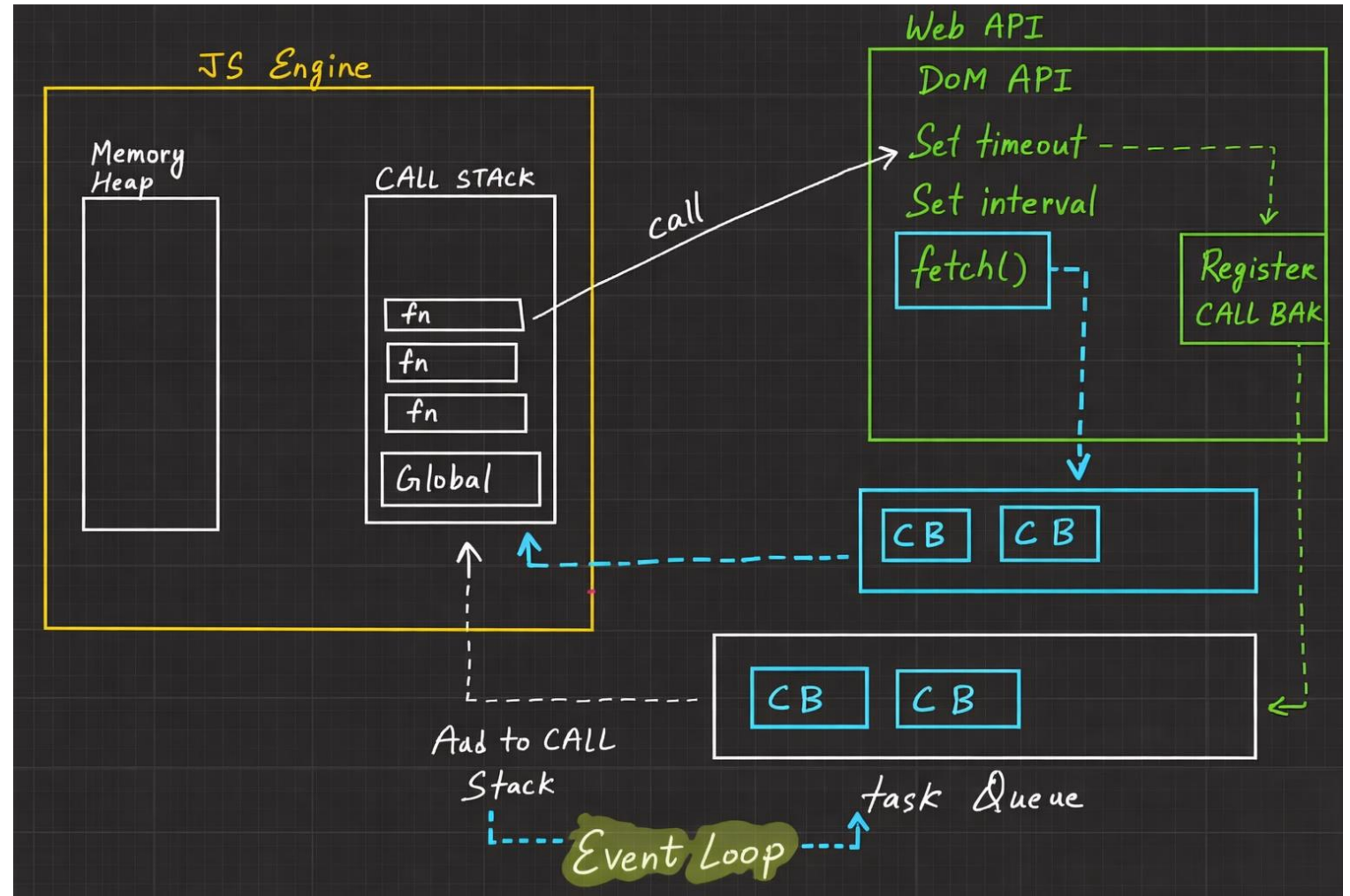


EVENT LOOP



Web Performance Metrics

+

Asset Optimization Techniques

+

Introduction to caching and CDNs

Web Performance Metrics

Measuring performance provides an important metric to help you assess the success of your app, site, or web service.

For example, the performance metrics to determine how your app performs compared to a competitor or compare your app's performance across releases. The metrics should be relevant to your users, site, and business goals.

Why Performance Metrics Matter?

- 1.User experience:** People abandon slow sites. Studies show that 53% of mobile users leave a site that takes longer than 3 seconds to load.
- 2.SEO benefits:** Google uses Core Web Vitals as ranking factors, so better performance can lead to better search visibility.
- 3.Conversion rates:** Faster sites convert better. Amazon found that each 100ms of latency cost them 1% in sales.
- 4.Accessibility:** Performance issues often hit users with older devices or slower connections hardest.

There are Core Web Vitals metrics that measure your website performance and affect your search engine rankings:

- The **First Paint (FP)** measures when the browser first starts rendering parts of the page. This includes empty boxes without any content.
- The **First Contentful Paint (FCP)** measures when content, like text or images, is first rendered by the browser.
- **Largest Contentful Paint (LCP)**: Measures how quickly the main page content renders
- **Interaction to Next Paint (INP)**: Measures the latency for a user interaction
- Cumulative Layout Shift (CLS)**: Measures the visual stability of the page
- TTFB is a metric that measures the time between starting navigating to a page and when the first byte of a response begins to arrive.

For more,
Visit the following links:
<https://web.dev/performance>

<https://nitropack.io/blog/website-performance-metrics/>

<https://medium.com/@ignatovich.dm/a-quick-guide-to-core-web-vitals-96ee4d8c1dfe>



FCP (First Contentful Paint)

FCP is a performance metric that measures the time it takes for a web page to display the first piece of content to the user after they initiate a page load. Specifically, FCP tracks how long it takes from when a user begins loading a page until the first bit of content—such as text, an image, or a background element—appears on the screen.

Largest Contentful Paint (LCP)

LCP is a metric that measures the time it takes for the largest piece of content to appear on a page. Pages with low LCP signal to users that your page is loading quickly.

Interaction to Next Paint (INP)

INP is a metric that measures how responsive a page is to user input. Pages with low INP signal to users that a page responds quickly when they interact with them, making your website feel more reliable and delightful to use.

Cumulative Layout Shift (CLS)

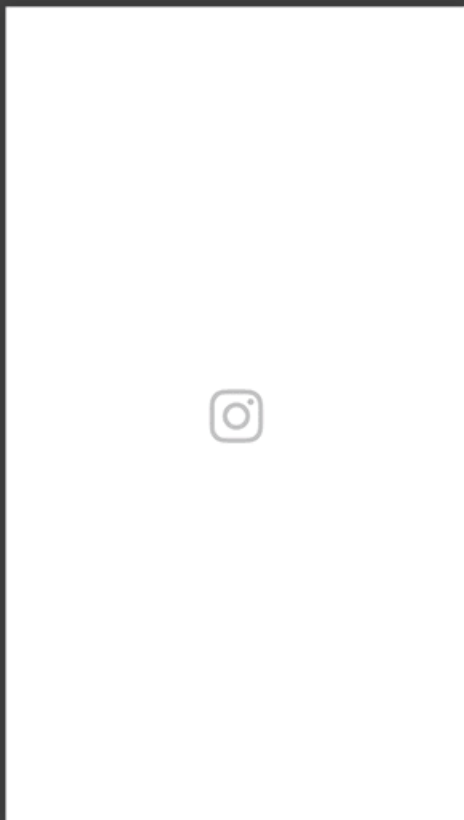
CLS is a metric that measures layout stability by observing the page for unexpected changes in the layout. Pages with low CLS signal to users that a page's layout is stable and won't change unexpectedly as they try to interact with it.

TTFB (Time to First Byte)

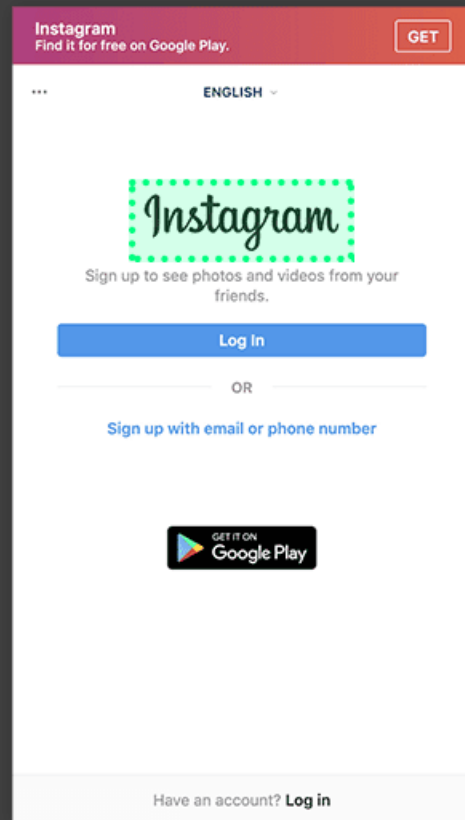
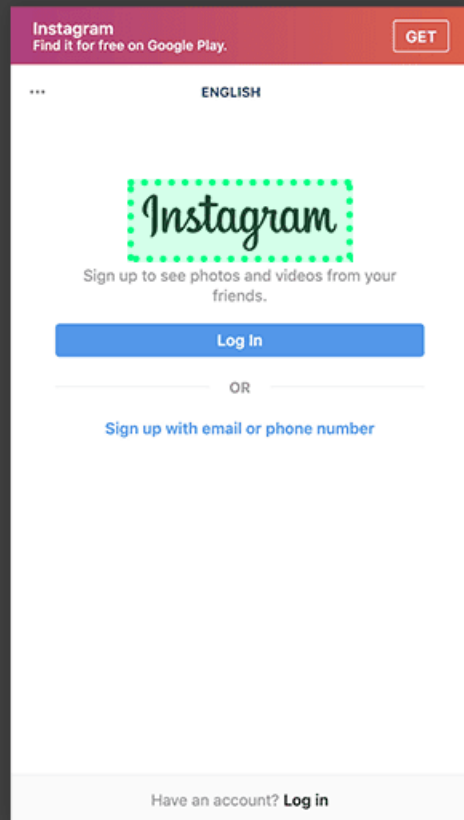
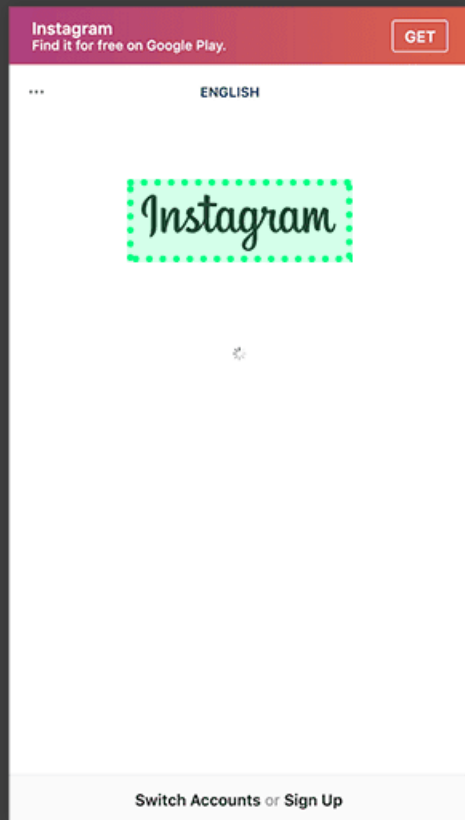
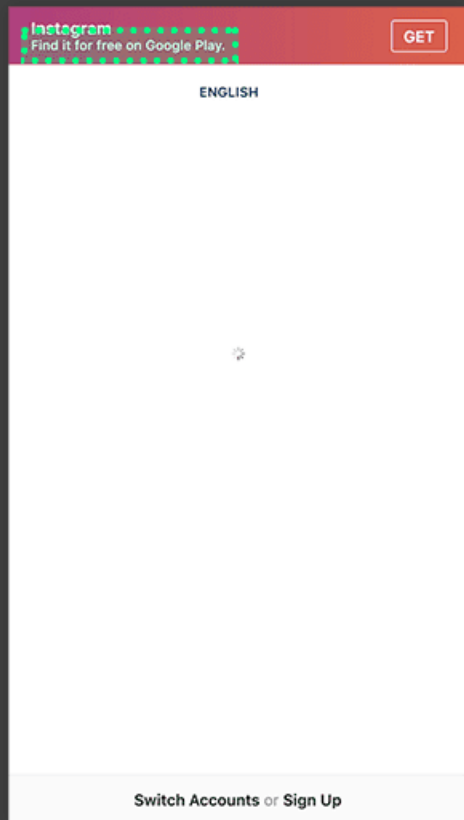
Time to First Byte (TTFB) measures how long it takes for a client's browser to receive the first byte of the response from the server. When a user tries to visit a page, their browser sends an HTTP request. The server that hosts the page has to process that request and return a response.

First Input Delay (FID)

First input delay (FID) measures the time from when a user first interacts with your site (for example, when they click a link, tap on a button, or use a custom, JavaScript-powered control) to the time when the browser is actually able to respond to that interaction.



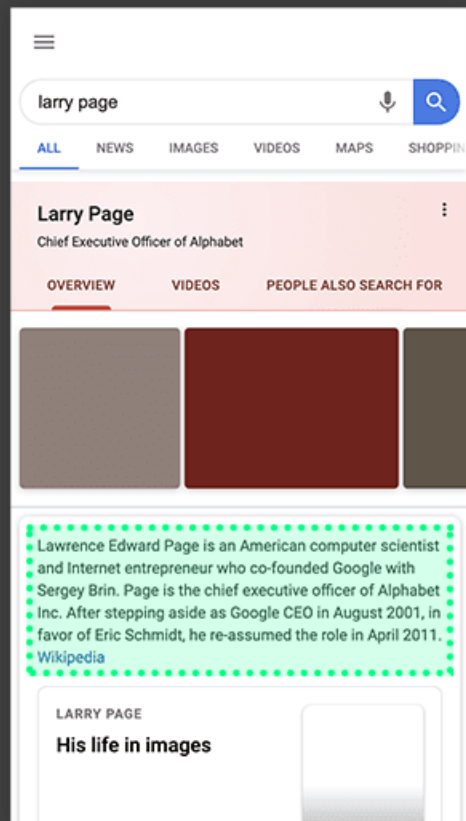
FCP



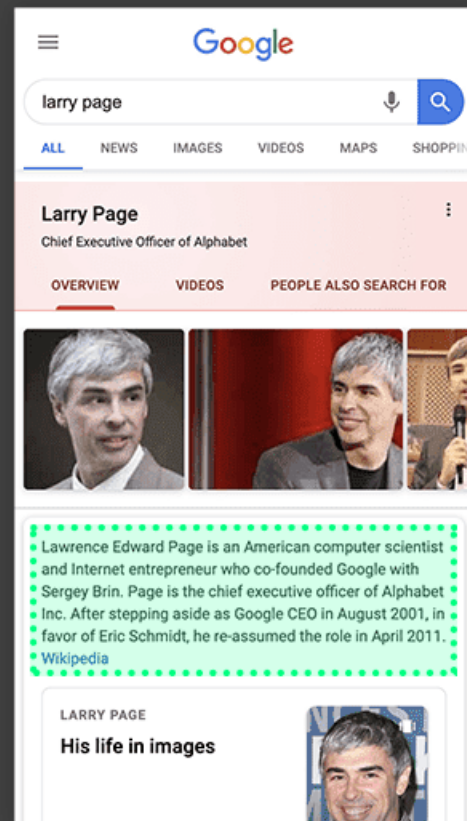
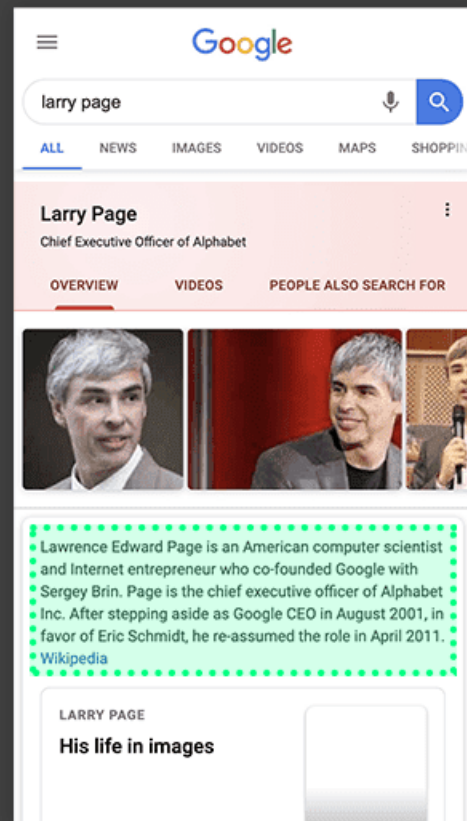
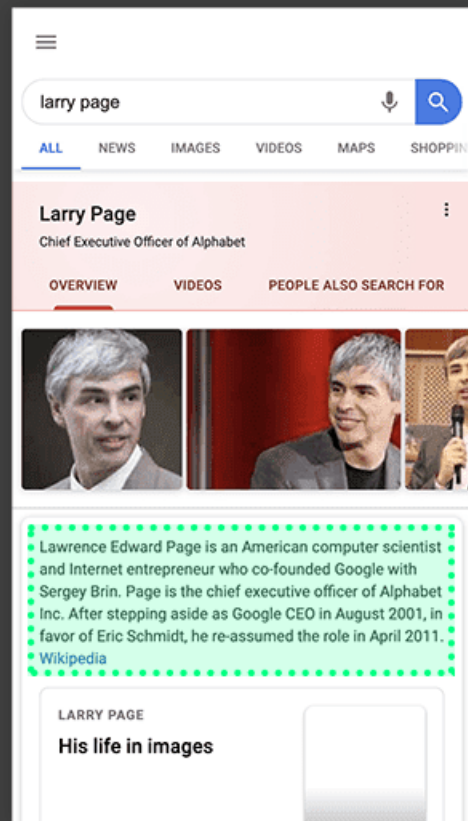
LCP



FCP



LCP



Tools for Measuring Performance

Lighthouse

Lab Tool

Built into Chrome DevTools. Audits performance, accessibility, SEO. Generates detailed reports with actionable suggestions.

Metric: 0–100 score

Chrome DevTools

Lab Tool

Network tab (waterfall), Performance tab (flame chart, FPS), Coverage tab (unused JS/CSS).

Metric: Real-time

PageSpeed Insights

Field + Lab

Combines lab data with real Chrome User Experience Report (CrUX) field data. Google's go-to public tool.

Metric: 0–100 + CrUX

WebPageTest

Lab Tool

Deep waterfall analysis, filmstrip view, multi-location testing, connection throttling.

Metric: Filmstrip

Techniques for Optimizing CSS and JavaScript

Heavy CSS and JavaScript files can slow down webpage loading.

CSS Optimization Techniques:

- Minify CSS (remove spaces and comments).
- Remove unused CSS rules.
- Use efficient selectors.

JavaScript Optimization Techniques:

- Minify JavaScript to reduce file size.
- Use 'defer' or 'async' attributes for scripts.
- Use code splitting to load only required scripts.

› JavaScript Optimization

Bundle Splitting / Code Splitting

Split your JS bundle into smaller chunks.
Load only what's needed per page.
Webpack, Rollup, Vite support this natively.

Tree Shaking

Eliminate dead code — functions and modules that are imported but never called.
Requires ES Modules (import/export syntax).

Lazy Loading Modules

Use dynamic import() to load heavy modules only when needed. Reduces initial bundle size significantly.

Defer & Async Scripts

Use `defer` to load scripts after HTML parsing. Use `async` for independent scripts. Never block the parser.

Minification & Compression

Terser removes whitespace, mangles variable names. Combine with Brotli/gzip compression on the server.

Web Workers

Offload heavy computation to a background thread. Keeps the main thread free for user interactions.

Minification

Minification is the process of removing unnecessary or redundant data without affecting how a resource is processed by the browser.

Minification can include the removal of code comments, white space, and unused code, as well as the shortening of variable and function names. Minification is used to improve web performance by reducing file size. It is generally an automated step that occurs at build time.

As minification makes code less legible to humans, developer tools have 'prettification' features that can add white space back into the code to make it a bit more legible.

Tree shaking

Tree shaking is a term commonly used within a JavaScript context to describe the removal of dead code.

It relies on the [import](#) and [export](#) statements to detect if code modules are exported and imported for use between JavaScript files.

In modern JavaScript applications, we use module bundlers (e.g., [webpack](#) or [Rollup](#)) to automatically remove dead code when bundling multiple JavaScript files into single files. This is important for preparing code that is production ready, for example with clean structures and minimal file size.

Code splitting

Code splitting is the practice of splitting the code a web application depends on — including its own code and any third-party dependencies — into separate bundles that can be loaded independently of each other. This allows an application to load only the code it actually needs at a given point in time, and load other bundles on demand. This approach is used to improve application performance, especially on initial load.

Code splitting is a feature supported by bundlers like [webpack](#) and [Browserify](#) which can create multiple bundles that can be dynamically loaded at runtime.

Lazy loading

Lazy loading is a strategy to identify resources as non-blocking (non-critical) and load these only when needed. It's a way to shorten the length of the [critical rendering path](#), which translates into reduced page load times.

Lazy loading can occur on different moments in the application, but it typically happens on some user interactions such as scrolling and navigation.

Lazy loading can be applied to multiple resources and through multiple strategies.

Examples include: code splitting, js script type=module, fonts using <link rel="preload">, images and iframes, Intersection Observer API

Asset Optimization

Asset Optimization (Images, Fonts, etc.)

Assets like images and fonts usually occupy the largest portion of webpage size.

Image Optimization Techniques:

- Compress images using tools such as TinyPNG.
- Use modern image formats like WebP.
- Implement lazy loading so images load only when needed.

Font Optimization Techniques:

- Limit the number of font families used.
- Use system fonts when possible.
- Load fonts efficiently.



TOPIC 03 › Asset Optimization



Image Optimization

Use modern formats

WebP saves 25–34% over JPEG; AVIF saves up to 50%. Always offer fallbacks.

Responsive images

Use srcset and sizes attributes. Serve appropriately sized images per device.

Lazy load images

Add loading="lazy" to . Browser defers off-screen image loading.

Compression

Use lossless for icons/logos (PNG), lossy for photos (WebP/JPEG at 80%).



Font Optimization

Use WOFF2 format

WOFF2 is 30% smaller than WOFF. Use font-display: swap to avoid invisible text.

Subset your fonts

Only include the characters you need. Google Fonts supports &subset= parameter.

Preload critical fonts

Add <link rel="preload" as="font"> for fonts used in above-the-fold content.

System font stack

Consider native system fonts for body text — zero download, great performance.

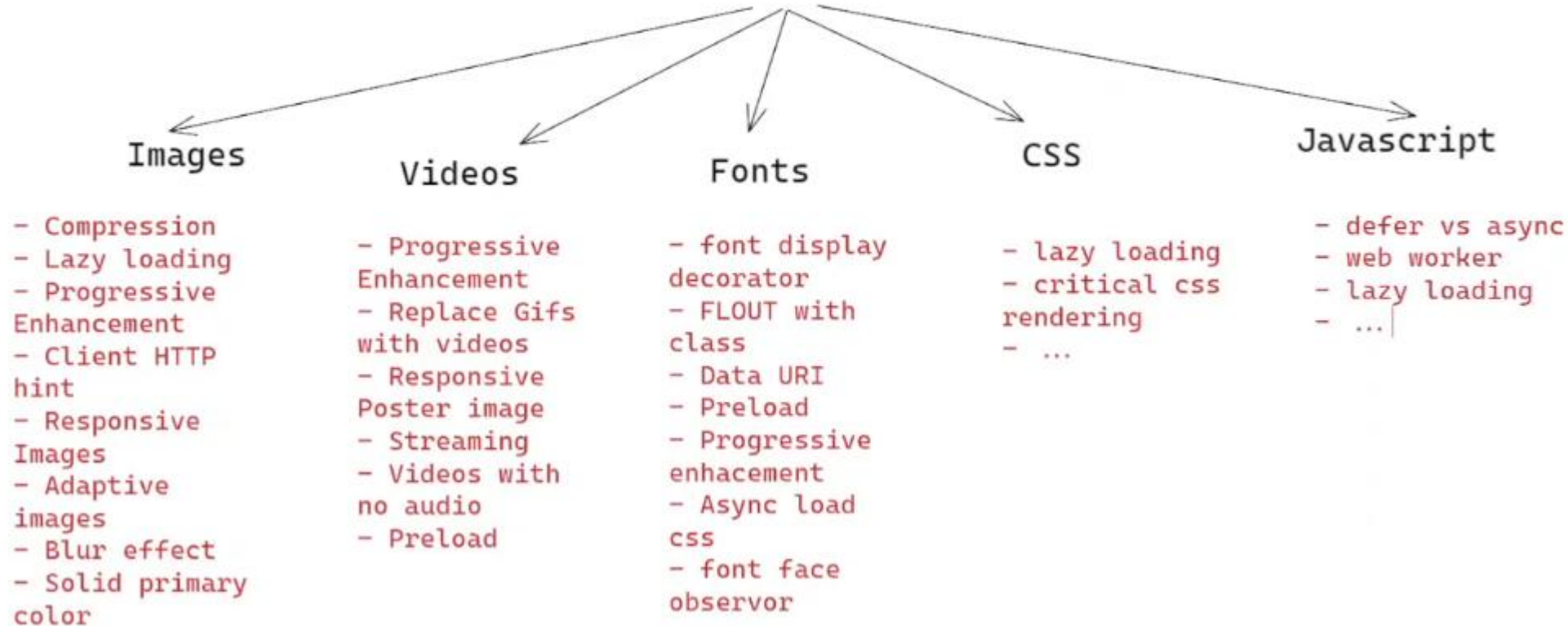
Image Format Comparison

Format	Best Use	Quality	File Size vs JPEG	Browser Support
JPEG	Photos	Good	Baseline	Universal
PNG	Icons, logos, transparency	Lossless	Larger	Universal
WebP	Photos & graphics	Better	25–34% smaller	All modern browsers
AVIF	Photos & graphics	Best	Up to 50% smaller	Chrome, Firefox, Safari 16+
SVG	Icons, illustrations, logos	Vector	Tiny (scalable)	Universal

💡 Pro tip: Use <picture> element with AVIF → WebP → JPEG/PNG fallbacks for maximum compatibility and performance.

Assets Optimization

Frontend Performance Optimization Techniques



For more on this refer to the link below:

<https://web.dev/learn/performance/optimize-resource-loading>



Images

- Compression** — reduce file size
- Lazy loading** — load images only when visible in viewport
- Progressive Enhancement** — start with low quality, improve as it loads
- Client HTTP hint** — browser tells server what it supports
- Responsive Images** — serve different sizes for different screens
- Adaptive images** — smarter version, considers network/device
- Blur effect** — show blurred placeholder while loading
- Solid primary color** — show dominant color as placeholder

CSS

- Lazy loading** — load non-critical CSS later
- Critical CSS rendering** — inline above-the-fold CSS to paint faster

JavaScript

- defer vs async** — control when scripts execute relative to HTML parsing
- Web Worker** — run JS off the main thread to avoid blocking UI
- Lazy loading** — load scripts only when needed

Videos

- Progressive Enhancement** — basic first, enhanced for capable browsers
- Replace GIFs with videos** — videos are far smaller than animated GIFs
- Responsive Poster image** — show a thumbnail before video loads
- Streaming** — don't wait for full download before playing
- Videos with no audio** — saves bandwidth when audio isn't needed
- Preload** — fetch video early before user clicks play

Fonts

- Font display decorator** — controls font swap behavior (`font-display: swap`)
- FLOUT with class** — Flash of Unstyled Text handled with CSS class toggling
- Data URI** — embed font directly in CSS
- Preload** — tell browser to fetch font early
- Progressive enhancement** — fallback fonts first
- Async load CSS** — load font CSS non-blocking
- Font Face Observer** — JS library to detect when font is ready

Progressive Enhancement

(Image Optimization)

AVIF as a Progressive Enhancement

```
<picture>
  <source srcset="photo.avif" type="image/avif">
  <source srcset="photo.webp" type="image/webp">
  
</picture>
```

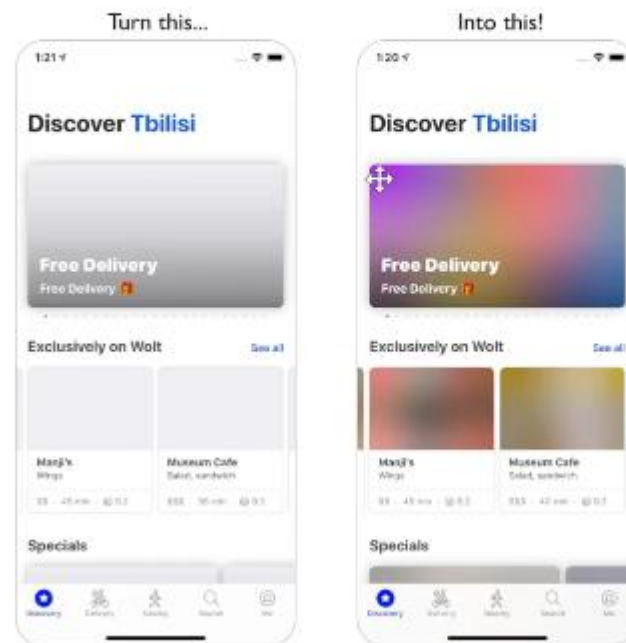
AVIF

WebP

Fallback JPEG

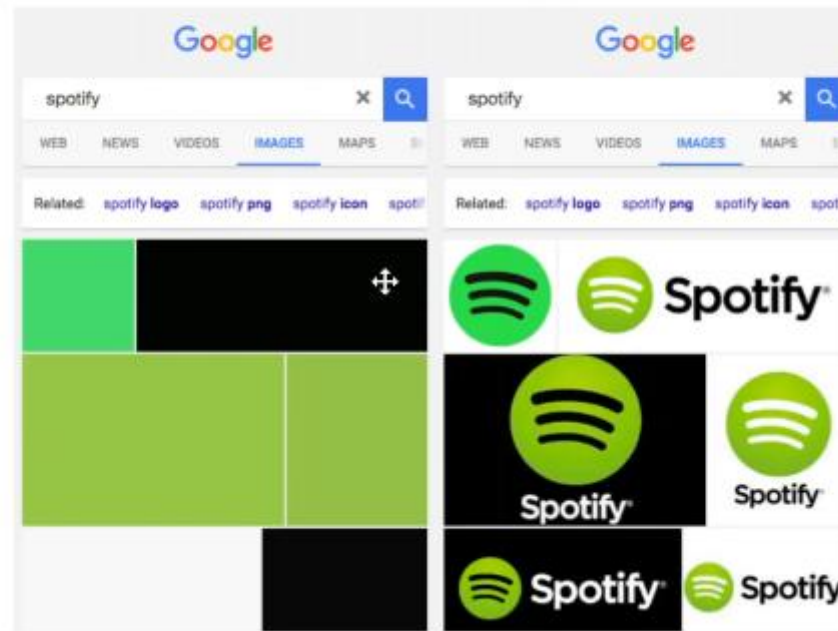
Small image with blur effect

(Image Optimization)



Solid Primary Color As Backgr

(Image Optimization)



↑ Google Images Search shows a solid background as placeholder (left image is while loading, right when already loaded).

Introduction to Caching

Caching stores frequently used data so it can be reused quickly without downloading again.

Types of Caching:

- Browser Caching – Stores resources like CSS, JS, and images locally in the browser.
- Server Caching – Stores results of frequently requested server operations.

Benefits of Caching:

- Faster page loading
- Reduced server load
- Better user experience

For more refer to

<https://www.cloudflare.com/learning/cdn/what-is-caching/>

Introduction to CDNs (Content Delivery Networks)

A CDN is a distributed network of servers that deliver website content from the nearest geographic location to the user.

How CDN Improves Performance:

- Reduces latency by serving content from nearby servers.
- Improves website loading speed.
- Distributes traffic across multiple servers.

Examples of CDN providers: Cloudflare, Akamai, Amazon CloudFront.

For more on CDN's refer to

<https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>

Best Practices for Web Performance Optimization

- Minify CSS and JavaScript files
- Compress and optimize images
- Use lazy loading for media
- Enable browser caching
- Use a CDN for faster content delivery
- Reduce unnecessary HTTP requests